

# Supplement to: Translating HOL to ACL2 with Set Theory

Matt Kaufmann<sup>1</sup>[0009–0004–5667–4888] and  
Konrad Slind<sup>2</sup>[0009–0007–9898–7518]

<sup>1</sup> `kaufmann@cs.utexas.edu`

<sup>2</sup> `konrad.slind@gmail.com`

**Abstract.** We present further details for a soundness argument in the paper, “Translating HOL to ACL2 with Set Theory” [1] by the present authors.

**Keywords:** Interactive Theorem Prover · ACL2 · HOL4.

## 1 Introduction

The paper, “Translating HOL to ACL2 with Set Theory” [1], presents a correctness result in Section 6. This Supplement fleshes out an argument for that result, sketched very briefly in that paper, into a more detailed sketch.

We assume familiarity with that paper, which we reference below as the *Main Paper*. In particular, we assume familiarity with Section 6 of the Main Paper, but here we summarize the notation and argument presented in that section. Even with this summary we do not intend that this Supplement be self-contained; it is just an extension of Section 6 of the Main Paper. After the brief summary below, we start where that section leaves off. We use the HOL4 theory `ex1` from Section 5.1 of the Main Paper to illustrate our argument sketch.

Before stating the claims from the Main Paper for which we argue below, we provide some key notation.

- $T'$  and  $G'$  arise from the respective translations of HOL theory  $T$  and HOL goal  $G$  by treating hypothesis functions and `ex1$prop` as returning `t`.
- $T_Z$  and  $G_Z$  replace the respective translations of  $T$  and  $G$  by translating the original explicit HOL definitions (not, for example, recursive “definitions”) and by trafficking in HOL values instead of HOL value pairs. Thus, the new unary functions return just a HOL value instead of a HOL value pair. As in the Main Paper, we do not define these straightforward translations explicitly. Also, they are not *implemented*; they are merely *logical*, to support our argument.
- If  $S$  is an ACL2(`zfc`) session or its theory, we write  $\text{ZFC}(S)$  to denote the following extension of ZFC.
  - The Comprehension and Replacement schemes of ZF apply to formulas that use function symbols of  $S$ .

- Included are all equations `(equal (h) t)` for hypothesis functions  $h$  of  $S$ .
- If  $S$  is an  $\text{ACL2}(\text{zfc})$  session or its theory, then  $[[S]]$  is the theory extending  $S$  with  $\text{ZFC}(S)$ .

Note that  $T_Z$  is formulated in  $\text{ACL2}(\text{zfc})$ , where set theory axioms are limited by hypothesis functions. Those limitations are removed in  $[[T_Z]]$ .

Here are the Claims from Section 6 of the Main Paper.

- (A) If  $[[T_Z]] \vdash G_Z$ , then  $T \models G$ .
- (B) If  $[[T']] \vdash G'$ , then  $[[T_Z]] \vdash G_Z$ .

We use the following goal  $G$  taken from `ex1` to sketch our argument.<sup>3</sup>

$$(1) \quad (\text{FST } x = \text{FST } y \wedge \text{SND } x = \text{SND } y) \Leftrightarrow (x = y)$$

The following  $\text{ACL2}(\text{zfc})$  definitions support the arguments below.

```
(hol-valuep x type hta) = (in x (hol-type-eval type hta))
```

```
(hpp p hta) = (and (cons p)
  (hol-typep (cdr p) hta)
  (hol-valuep (car p) (cdr p) hta))
```

## 2 Argument for Claim (A).

Let  $\text{apply}(f, v)$  denote the set-theoretic application of a function  $f$  (a set of ordered pairs) to  $v$ , while  $\text{apply}^*$  abbreviates iterated application; for example,  $\text{apply}^*(f, a, b)$  is the curried application of  $f$  to  $a$  and  $b$ , that is,  $\text{apply}(\text{apply}(f, a), b)$ . We assume constant symbols **hol-and**, **hol-equal**, **hol-fst**, and **hol-snd** with suitable axioms. Then the Pitts semantics produces the following set-theoretic expression to formalize the meaning of (1), where variables  $x$  and  $y$  are implicitly universally quantified.

$$(2) \quad \text{apply}^*(\text{hol-equal}, \\ \text{apply}^*(\text{hol-and}, \\ \text{apply}^*(\text{hol-equal}, \text{apply}(\text{hol-fst}, x), \text{apply}(\text{hol-fst}, y)), \\ \text{apply}^*(\text{hol-equal}, \text{apply}(\text{hol-snd}, x), \text{apply}(\text{hol-snd}, y))), \\ \text{apply}^*(\text{hol-equal}, x, y))$$

To produce formulas such as  $T_Z$  and  $G_Z$  we transform each *term* expressible in set theory, such as (2), to a *formula* of set theory, more specifically a formula of  $\text{ACL2}(\text{zfc})$ . This involves equating (2) with the HOL constant, `TRUE` (which is of course definable in  $\text{ACL2}(\text{zfc})$ ). But we also need to account for the fact that HOL functions have limited domains. For example, the domain of `FST` is the Cartesian product of two arbitrary monomorphic HOL types, which thus does not contain (for example) the number 3. In  $\text{ACL2}(\text{zfc})$  we have defined `hol-typep` to recognize valid monomorphic types. The  $\text{ACL2}(\text{zfc})$  binary

<sup>3</sup> Technically  $G$  is a HOL *term*, but Boolean HOL terms may be viewed as formulas.

function, `in`, denotes set membership; that is, `(in x s)` is a term corresponding to the formula,  $x \in s$ . These functions let us formalize the domain restrictions in ACL2(zfc) as formula (3H) below, while formula (3C) equates the term (2), displayed above, to the representation of HOL TRUE in ACL2(zfc), `(hol-true)`. So the ACL2(zfc) formula  $G_Z$  corresponding to term (2) is the implication

$$(3) \quad (3H) \implies (3C)$$

where (3H) and (3C) are as follows. (See the Main Paper for discussion of the [...] abbreviation for list construction.)

```
(3H) (and (alist-subsetp (ex1$hta) hta)
          (hol-typep a hta)
          (hol-typep b hta)
          (in x (hol-type-eval [:hash a b] hta))
          (in y (hol-type-eval [:hash a b] hta)))
(3C) (equal (apply* (hol-equal)
                   (apply* (hol-and)
                           (apply* (hol-equal)
                                   (apply (hol-fst) x)
                                   (apply (hol-fst) y))
                           (apply* (hol-equal)
                                   (apply (hol-snd) x)
                                   (apply (hol-snd) y))))
      (hol-true))
```

Here is our justification for Claim (A).

- (a) Goal: Show  $T \models G$ .
- (b) It suffices to show that  $ZFC \vdash (T \models G)$ , since any formal set-theory proof can be turned into an informal mathematical proof.
- (c) It suffices to show that  $ZFC(T_Z) \vdash (T \models G)$ , since  $ZFC(T_Z)$  is a conservative extension of ZFC (as it is contained in a definitional extension, by defining the hypothesis functions in  $T_Z$  to be true and defining the other function symbols of  $T_Z$  arbitrarily).
- (d) It suffices to show that  $ZFC(T_Z) \vdash (\overline{T_Z} \implies \overline{G_Z})$ . Explanation is below.
- (e) It suffices that  $ZFC(T_Z) \cup T_Z \vdash G_Z$ , by the Deduction Theorem of first order logic.
- (f) That concludes the proof, since  $[[T_Z]]$  is the extension of  $T_Z$  with  $ZFC(T_Z)$ .

Here is a plausibility argument for Step (d) above, working in the set theory  $ZFC(T_Z)$ . Consider the straightforward mapping that takes a type assignment  $A$  (a function from type variables to sets), supporting the Pitts semantics, to a corresponding HOL type-alist  $A_Z$ , so that a mapping of type  $t$  to set  $s$  in  $A$  corresponds to a pair  $\langle t, s \rangle$  in  $A_Z$ . The following property is surely plausible.

- (\*) For any HOL formula  $P$  and assignment  $A$  that covers the type variables of  $P$ ,  $\overline{P}$  holds in the model  $M$  under  $A$  if and only if  $\overline{P_Z}$  holds under  $A_Z$ .

So — again, working in  $\text{ZFC}(T_Z)$  — assume  $(\overline{T_Z} \implies \overline{G_Z})$  and that  $\overline{T}$  holds in the model  $M$  under an assignment  $A$  of the type variables in  $T$  and  $G$ , to show  $\overline{G}$  holds in  $M$  under  $A$ . Since  $\overline{T}$  holds in  $M$  under  $A$ , then  $\overline{T_Z}$  holds under  $A_Z$  by (\*). By  $(\overline{T_Z} \implies \overline{G_Z})$ ,  $\overline{G_Z}$  holds under  $A_Z$  as well; hence  $\overline{G}$  holds in  $M$  under  $A$  by (\*).  $\square$

### 3 Argument for Claim (B).

We wish to deduce  $[[T_Z]] \vdash G_Z$  from  $[[T']] \vdash G'$ . Below we introduce a theory  $T''$  that is derived from  $T_Z$  but, like  $T'$ , traffics in HOL value pairs. The idea is, very roughly speaking, to use  $T''$  to give us  $G_Z$ . We'll be more precise below; for now, we start by defining  $T''$ .

Working in  $\text{ACL2}(\text{zfc})$ : For each unary function symbol  $(f_Z \text{ typ})$  defined in  $T_Z$ , we define a corresponding unary function symbol  $(f \text{ typ})$  but which traffics in HOL value pairs instead of HOL values. For example, if the translation of **FST** in  $T_Z$  is  $f_Z$ , then  $(f_Z \text{ [:arrow [:hash a b] a]})$  returns a set-theoretic function that maps an input  $\langle x, y \rangle$  (which has type  $[:\text{hash a b}]$ ) to  $x$  (which has type  $\text{a}$ ), while  $(f \text{ [:arrow [:hash a b] a]})$  instead returns a set-theoretic function that maps an input  $\langle \langle x, a \rangle, \langle y, b \rangle \rangle$  to  $\langle x, a \rangle$ . A bit more generally, if  $f_Z$  is the translation of a function of the same type as **FST**, then we have the following.

$$\begin{aligned} & (\text{apply } (f \text{ [:arrow [:hash a b] a]}) \langle \langle x, a \rangle, \langle y, b \rangle \rangle) \\ & = \\ & \langle (\text{apply } (f_Z \text{ [:arrow [:hash a b] a]}) \langle x, y \rangle), a \rangle \end{aligned}$$

Let  $T''$  be the set of all resulting definitional equations, *i.e.*, defining each  $f$  as suggested above as  $f_Z$  ranges over the functions defined in  $T_Z$ . We assume that for any HOL function whose definition is being translated from  $T$ , the same function symbol is introduced for the translation to  $T'$  as for the translation to  $T''$ .

Write  $T'$  as the disjoint union  $S \cup T'_0$ , where  $S$  is the (theory of the) session obtained by including the translator code (including, for example, the definition of **hpp** displayed earlier above) and  $T'_0$  is derived from the events introduced by translating  $T$ . Note that  $T_Z$  is a definitional extension of  $S$ , so below, the union  $T'' \cup T_Z$  is equal to  $S \cup T'' \cup T_Z$ . Also note that for each  $f$  defined in  $T'_0$  (equivalently, in  $T''$ ),  $f$  does not occur in  $[[S]]$ .

To prove Claim (B), we assume

$$(B1) \quad [[T']] \vdash G'$$

to show  $[[T_Z]] \vdash G_Z$ . For our example of **ex1**,  $G_Z$  is displayed in (3) above and  $G'$  can be displayed analogously as follows.

$$(4) \quad (\text{implies } (4H) \ (4C))$$

where

```

(4H) (and (alist-subsetp (ex1$hta) hta)
        (hpp p hta)
        (equal (hp-type p) [:hash a b]))
      (hpp q hta)
      (equal (hp-type q) [:hash a b]))
(4C) (equal (hp= (hp-and
                (hp= (hap* (fst [:arrow [:hash a b] a]) p)
                    (hap* (fst [:arrow [:hash a b] a]) q))
                (hp= (hap* (snd [:arrow [:hash a b] b]) p)
                    (hap* (snd [:arrow [:hash a b] b]) q)))
            (hp= p q))
      (hp-true))

```

In  $[[T'' \cup T_Z]]$  we can obtain  $G_Z$  from  $G'$ , i.e., (3) from (4), by instantiating variables  $p$  and  $q$  of (4) to suitable terms of (3):  $\langle x, [:hash\ a\ b] \rangle$  and  $\langle y, [:hash\ a\ b] \rangle$ , respectively. Thus we have

$$(B2) \quad [[T'' \cup T_Z]] \vdash \overline{G'} \implies \overline{G_Z}.$$

By a similar argument applied to the imported definitional formulas of  $T'_0$ , and noting that the type lemmas of  $T'_0$  hold trivially, we obtain (recalling that  $T_Z$  contains  $S$  and  $T'$  is  $S \cup T'_0$ )

$$(B3) \quad [[T'' \cup T_Z]] \vdash T'.$$

It follows from (B1) and (B3) that  $[[T'' \cup T_Z]] \vdash G'$ , and hence by (B2),

$$(B4) \quad [[T'' \cup T_Z]] \vdash G_Z.$$

But  $[[T'' \cup T_Z]]$  is a definitional extension of  $[[T_Z]]$  — the extra Comprehension and Replacement axioms of  $T''$  are of course derivable from  $T_Z$  since  $T''$  is a definitional extensions of  $[[T_Z]]$ . So by (B4) and conservativity of definitional extensions,  $[[T_Z]] \vdash G_Z$ , as was to be shown.  $\square$

**Acknowledgments.** We thank Rob Arthan for his feedback on a draft of the Main Paper that included an earlier version of this Supplement.

## References

1. Kaufmann, M., Slind, K.: Translating HOL to ACL2 with set theory. In: Nipkow, T., Passmore, G., Sultana, N. (eds.) *Logic, Computation and Proof*, vol. 16870. Springer Nature Switzerland (2026)